中南林业科技大学

编译原理实验报告

课程名称：　　编译原理

专业班级：

姓　　名：

学　　号：

完成日期：2026 年 10 月

目　录

实验一　词法分析 1
一、实验目的. 1
二、实验内容. 1
三、实验理论依据. 1
四、测试数据及运行结果展示. 5
结　论. 7

实验一 词法分析

一、 实验目的

编制一个读单词过程，从输入的源程序中识别出各个具有独立意义的单词，即基本保留字、标识符、常数、运算符、分隔符五大类，并依次输出各个单词的内部编码及单词符号自身值。

通过本次实验，掌握词法分析器的设计方法：先写出语言的词法规则，把词法规则转换为状态转换图，再把各转换图的初态连接在一起构成识别该语言的自动机，最后据此设计扫描器。扫描器作为语法分析的一个过程，当语法分析需要一个单词时被调用，它从初态出发，识别一个单词后进入终态，送出一个二元式。

二、 实验内容

（1）确定单词的分类及内部编码，明确五大类单词各自包含哪些符号。

（2）写出 C 语言子集的词法规则（正规式），包括标识符、常数、运算符、分隔符、注释与空白。

（3）绘制各类单词识别的状态转换图，合并初态构成识别该语言全部单词的自动机。

（4）用 C++ 实现扫描器程序，采用“最长匹配＋超前搜索一个字符”的策略，支持注释与预处理指令的跳过，并对非法输入给出错误定位。

（5）用实验指导书给出的示例程序与测试数据运行程序，验证输出与预期结果一致。

实验环境：Windows 11 操作系统，g++（MinGW-w64）8.1.0 编译器，C++17 标准；源程序为单文件 lexer.cpp。

三、 实验理论依据

1．词法分析的任务

词法分析是编译过程的第一个阶段（扫描阶段）。它的输入是源程序的字符

流，输出是单词符号（token）序列。每个单词用一个二元式表示：（单词种别，单词符号的属性值）。其中单词种别即该单词的内部编码，通常用整数表示；属性值即单词符号自身值，如标识符的名字、常数的数值、运算符的字符串等。本程序输出的二元式形如 (1,"int")、(3,"-5")。

2．单词的分类与内部编码

按照实验要求，把所有单词分成五大类，并分别赋予 1～5 的内部编码，具体见表 1。

表 1　单词的分类与内部编码

编码	单词类别	说　明	示　例
1	基本保留字	语言中有固定含义的关键字	int　float　if　else
2	标识符	用户定义的名字	main　a　y1　scanf
3	常数	整型、实型、字符串、字符	4　-5　3.1416　"%d"
4	运算符	算术、关系、逻辑、赋值等	=　+　>=　&&　++　<<=
5	分隔符	起分隔与定界作用	(　)　{　}　,　;　:

3．词法规则

记 letter 为任一字母，digit 为任一十进制数字，hexdigit 为任一十六进制数字，则本语言各单词的词法规则可用正规式描述，见表 2。

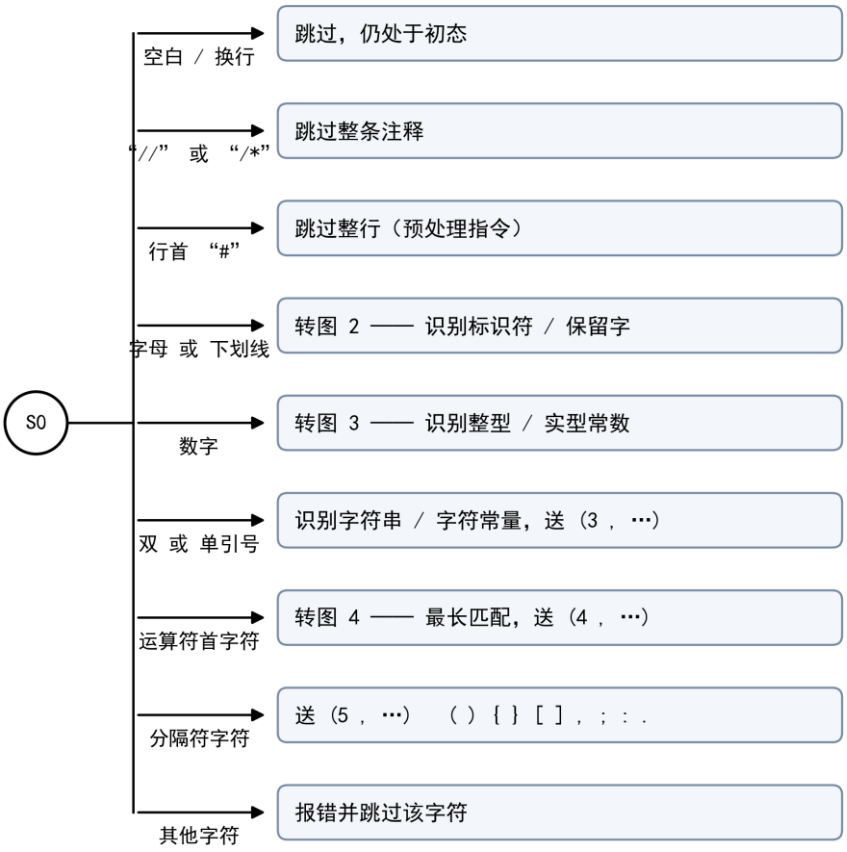
表 2　各单词的词法规则

单词类别	正规式（或说明）
标识符	letter (letter | digit | _)*
保留字	保留字表中的某一项，共 37 个
无符号整数	digit digit*　　十六进制：0 (x | X) hexdigit+
实型数	digit* . digit+ ((e | E) (+ | -)? digit+)?
字符串常数	" (除引号与换行的字符 | 转义序列)* "
字符常数	' (除引号与换行的字符 | 转义序列)* '
运算符	+ - * / % = < > ! & | ^ ~ ? 及其双字符、三字符组合，按最长匹配
分隔符	( ) { } [ ] , ; : .
注释	// 至行末；/* 任意字符序列 */　　——扫描时跳过
空白与预处理	空格、制表、回车、换行；行首 # 至行尾　——扫描时跳过

4．状态转换图

按照实验步骤，先分别画出各类单词识别的状态转换图，再把各转换图的初

态连接在一起，得到识别该语言全部单词的自动机。合并后的主控状态转换图如图 1 所示。



注：各分支识别出一个单词后均进入终态，送出二元式并回到 S0。

图 1　合并初态后的主控状态转换图

图 1 中，状态 S0 是一切单词识别的初态，程序每识别出一个单词、送出一个二元式后都返回 S0；遇到空白、注释和预处理指令时直接跳过，不产生单词。各类单词的具体识别过程如图 2～图 4 所示。

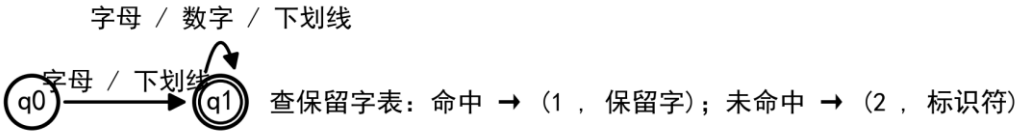


图 2　标识符与保留字的识别

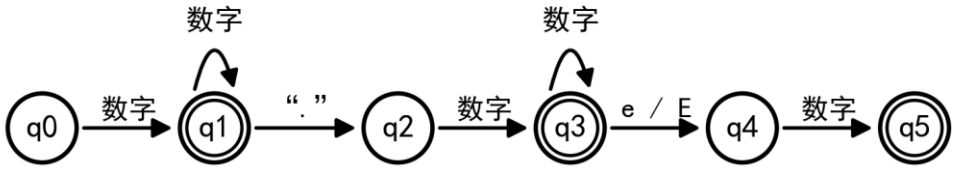


图 3　常数的识别

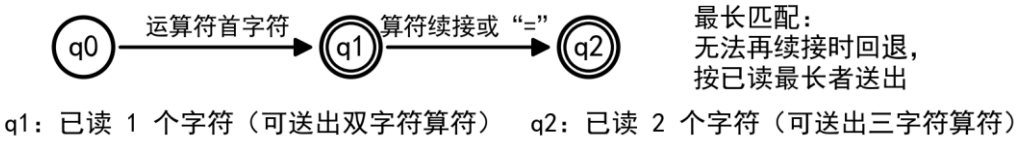


图 4　运算符的识别（最长匹配）

5．由状态转换图到确定的有限自动机

上述各状态转换图中，任一状态下对同一个输入字符最多只有一条转移，即状态转移互不冲突，因此由状态转换图直接得到的就是一个确定的有限自动机（DFA），无需再进行子集构造（确定化）和最小化处理。程序中用一重循环加分支判断实现该自动机：读入一个字符后，按当前所处的识别路径决定后继状态，路径结束即到达终态，此时回送二元式。

为避免把一个单词的前缀误当作整个单词（例如把 >= 拆成 > 和 =、把 3.14 拆成 3 和 .14），扫描器采用最长匹配原则：到达终态后并不立即送出，而是再向前看一个字符，若该字符仍可续接则继续转移，直到不能再续接为止，然后回退这个“多看”的字符并送出当前已识别的最长单词。这就是词法分析中的超前搜索。

6．保留字与标识符的同构处理

从正规式看，所有保留字都满足标识符的构成规则，即保留字集合是标识符集合的子集。因此二者可以共用同一条识别路径：先按标识符识别出一个字符串，再查保留字表，命中则为保留字（编码 1），否则为标识符（编码 2）。程序中用哈希表（unordered_set）存放 37 个保留字，平均查找时间为 O(1)。这样既省

去了为保留字单独设计状态转换图，也避免了保留字被误识别为标识符。

7．正负号的归属

字符“-”在源程序中有两种含义：一是双目算术减号，属于运算符；二是常量的符号位。若不加区分，int a=-5 中的 -5 就会被拆成 (4,"-") 和 (3,"5")，与要求输出不符。为此程序设置了一个状态变量 operandExpected，记录“下一个位置是否期待一个运算对象”：在初态、运算符之后，以及 ([{ , ; : 等分隔符之后置为真；在标识符、常数、)] 之后置为假。只有当 operandExpected 为真且正负号后紧跟数字时，才把符号并入常数。于是 a=-5 输出 (3,"-5")，而 a-b 输出 (4,"-")。

8．注释、预处理指令与错误处理

注释不是单词，扫描时直接跳过；块注释允许跨行，并检查是否以 */ 闭合。行首的 # 指令（如 #include "math.h"）不属于五大类单词，若逐个字符识别会把 math 和 h 误当作标识符，故整行跳过并单独提示。遇到无法识别的字符、未闭合的注释或字符串、非法常数（如 123abc）时，程序给出其行号、列号，跳过该字符后继续扫描，使一次运行能够报出全部词法错误。

四、 测试数据及运行结果展示

（一）测试数据一

取自实验指导书给出的示例程序，源程序如下：

```
main()
{
int  a=-5,b=4,j;
if(a>=b)
j=a-b;
else  j=b-a;
}
```

程序运行后输出的二元式序列如下：

```
(2,"main")  (5,"(")  (5,")")  (5,"{")  (1,"int")  (2,"a")
(4,"=")  (3,"-5")  (5,",")  (2,"b")  (4,"=")  (3,"4")  (5,",")
(2,"j")  (5,";")  (1,"if")  (5,"(")  (2,"a")  (4,">=")  (2,"b")
(5,")")  (2,"j")  (4,"=")  (2,"a")  (4,"-")  (2,"b")  (5,";")
```

```
(1,"else")  (2,"j")  (4,"=")  (2,"b")  (4,"-")  (2,"a")
(5,";")  (5,"}")
```

将上述输出与实验指导书给出的“要求输出”逐项对照，单词的类别编码、出现顺序及单词符号自身值完全一致，其中 -5 作为一个常数、>= 作为一个运算符、main 作为标识符处理。

（二）测试数据二

实验指导书给出的测试数据包含预处理指令、实型常数、字符串常数、地址运算符与逻辑运算符，源程序如下：

```
#include "math.h"

main()
{
float a,b,y1,y2;
int c,d,e;
a=3.1415926;b=25.0;
scanf("%d,%d",&c,&d);
y1=cos(a/3);
y2=sqrt(b);
e=d+(-1);
if(c>d&&c>0)  e=c;
printf("y1=%f,y2=%f,e=%d,c=%x\n",y1,y2,e,c);
}
```

程序运行后输出的二元式序列如下（预处理指令行已跳过）：

```
(2,"main")  (5,"(")  (5,")")  (5,"{")  (1,"float")  (2,"a")
(5,",")  (2,"b")  (5,",")  (2,"y1")  (5,",")  (2,"y2")  (5,";")
(1,"int")  (2,"c")  (5,",")  (2,"d")  (5,",")  (2,"e")  (5,";")
(2,"a")  (4,"=")  (3,"3.1415926")  (5,";")  (2,"b")  (4,"=")
(3,"25.0")  (5,";")  (2,"scanf")  (5,"(")  (3,"%d,%d")  (5,",")
(4,"&")  (2,"c")  (5,",")  (4,"&")  (2,"d")  (5,")")  (5,";")
(2,"y1")  (4,"=")  (2,"cos")  (5,"(")  (2,"a")  (4,"/")
(3,"3")  (5,")")  (5,";")  (2,"y2")  (4,"=")  (2,"sqrt")
(5,"(")  (2,"b")  (5,")")  (5,";")  (2,"e")  (4,"=")  (2,"d")
(4,"+")  (5,"(")  (3,"-1")  (5,")")  (5,";")  (1,"if")  (5,"(")
(2,"c")  (4,">")  (2,"d")  (4,"&&")  (2,"c")  (4,">")  (3,"0")
(5,")")  (2,"e")  (4,"=")  (2,"c")  (5,";")  (2,"printf")
(5,"(")  (3,"y1=%f,y2=%f,e=%d,c=%x\n")  (5,",")  (2,"y1")
(5,",")  (2,"y2")  (5,",")  (2,"e")  (5,",")  (2,"c")  (5,")")
(5,";")  (5,"}")
```

（三）运行结果分析

两组测试数据运行时均未报告词法错误。对其中若干易错片段的识别结果及原因分析见表 3。

表 3　易错片段的识别结果分析

源程序片段	识别结果	说　明
#include "math.h"	整行跳过	预处理指令不属五大类单词，避免 math 被误判
float、int	(1,"float")、(1,"int")	命中保留字表，编码为 1
3.1415926、25.0	(3,"3.1415926")、(3,"25.0")	小数点并入常数，不单独切分
"%d,%d"	(3,"%d,%d")	字符串整体为一个常数，% 与 , 不拆开
d+(-1)	(3,"-1")	处于期待运算对象的位置，符号并入常数
c>d&&c>0	(4,">")、(4,"&&")、(4,">")	最长匹配,&& 不被拆成两个 &
a/3	(4,"/")	/ 后不是 / 或 *,按运算符处理而非注释
"…c=%x\n"	整个字符串	转义字符不截断字符串

对测试数据二的统计结果为：保留字 3 个，标识符 31 个，常数 7 个，运算符 13 个，分隔符 37 个，合计 91 个单词。其中 main、scanf、printf、cos、sqrt 等均按标识符处理，符合 C 语言的词法规则。

结　论

本次实验按照“词法规则→状态转换图→自动机→扫描器”的步骤，用 C++ 实现了一个能识别 C 语言子集五大类单词的词法分析器。程序以 next() 作为供语法分析调用的扫描过程，把基本保留字、标识符、常数、运算符、分隔符分别编码为 1～5，按二元式依次输出，并附带行号、列号与分类统计。用实验指导书给出的示例程序与测试数据验证，输出结果与要求完全一致，未发现词法错误。

实验中的主要难点是边界情况的处理。其一是正负号的归属：减号既可以是运算符，也可以是常量的符号位，只有根据上下文判断当前位置是否期待一个运算对象，才能得到 (3,"-5") 与 (4,"-") 两种正确结果。其二是实型常数与成员访问运算符“.”的区别：只有小数点后紧跟数字时才并入常数。其三是

#include 这类预处理指令，必须整行跳过，否则其中的文件名会被拆成若干标识符。这些情况都说明，词法规则写得再完备，也要在实现时把“向前多看一眼”的边界判断落实到位。

通过本次实验，对最长匹配（超前搜索）这一词法分析的核心思想有了更具体的认识：自动机到达终态并不意味着单词就此结束，还必须再看一个字符才能确定单词的边界。同时也体会到，把词法规则形式化地写成正规式和状态转换图以后，程序的每一段分支都能与图中的一条路径对应，代码结构清晰、便于查错；若要增加新的运算符或关键字，只需修改相应的表，而不必改动识别逻辑。